FICHE MODULES 0 ET 1

Pourquoi changer, et la roue du PDLC

Pourquoi le cycle de delivery change maintenant, ce qui est déjà acté, et le modèle du nouveau cycle : six phases, une spec à chaque passage, et des niveaux de délégation qui disent qui tient le verdict.

Ce qui marche déjà, et ce qui coince à l'échelle

Module 0 · slide « Ce qui marche déjà, et ce qui coince à l'échelle »

Beaucoup d'équipes ont déjà de bonnes pratiques avec l'IA. Ce qui coince, c'est le passage à l'échelle :

Ce qui coince	Ce qui y répond
Chacun dans son couloir : chaque équipe avance avec son propre cycle	La roue du PDLC (module 1)
Des DoR / DoD qui varient : « prêt » et « fini » ne veulent pas dire la même chose partout	Les quality gates (module 2)
Une qualité qui tient localement : elle repose sur des personnes	Les quality gates (module 2)
Un contexte dans les têtes : précieux, mais invisible pour un nouvel arrivant et pour un agent	Le Context Nexus (module 3)
Des réussites IA locales : de belles trouvailles qui ne se partagent pas	Les rôles et la toolkit (modules 4 et 5)
Une valeur difficile à montrer : on sent le gain, on a du mal à le prouver	La mesure (module 6)

> Les bonnes pratiques existent déjà : le nouveau cycle sert à les partager.

Pourquoi maintenant

Module 0 · slide « Pourquoi maintenant »

Trois marches :

1. **Productivité individuelle** : chacun gagne du temps avec son assistant. Beaucoup d'équipes y sont déjà.
2. **Cas d'usage connectés** : l'IA suit le travail d'une phase à l'autre, avec un contexte partagé. C'est le pilote.
3. **Processus et organisation** : le cycle, les rôles et la mesure s'alignent sur ce nouveau mode. C'est le passage à l'échelle.

L'expérimentation a fait ses preuves ; reste à la mener ensemble, à l'échelle.

Ce qui est acté, ce qui se cale avec vous

Module 0 · slide « Ce qui est acté, ce qui se cale avec vous »

Acté	À caler pendant le pilote, avec les retours des équipes
La roue en six phases	Le curseur de délégation à l'IA, étape par étape
Des gates G1 à G4, passées sur preuve	La gate de bout en bout
Quatre casquettes dans chaque équipe	Les seuils des gates, calibrés chaque mois
Pendant le pilote, un humain valide chaque tâche	Le détail des indicateurs
Claude Code, Spec Kit et LiteLLM comme outils de référence	Le calendrier de chaque vague

Le fil rouge : Vélora

Module 0 · slide « Le fil rouge : Vélora »

Vélora, groupe hôtelier fictif, veut vendre le late check-out en ligne. Quatre équipes interdépendantes, reliées par des contrats :

- **Offre** : qui peut partir plus tard, et quand ; publie l'éligibilité.
- **Prix** : le prix, entre un plancher et un plafond.
- **Paiement** : en ligne ou à l'hôtel, en périmètre sensible ; dépend de l'offre et du prix.
- **Expérience** : la tuile dans l'application, jusqu'à l'achat ; dépend de l'offre et du prix.

Les exemples du module 3 prennent une autre équipe du même groupe : l'équipe Notifications et son service d'envoi VelNotif.

Une roue en six phases

Module 1 · slide « Une roue en six phases »

Phase	La question
Discover	Est-ce que ça vaut la peine ?
Design	Quelle solution, quels critères ?
Develop	Le code et ses tests
Deploy	En production, observable
Deepen	La valeur est-elle au rendez-vous ?
Drop	Retirer ce qui ne sert plus

La boucle se referme :

- Deepen rend un seul verdict, à partir d'une seule analyse (lecture métier et lecture technique ensemble) : continuer, repartir en Discover pour faire évoluer, ou retirer. « Continuer » est aussi une décision, et elle se trace.

- Drop non plus n'est pas une fin : le retrait repart en Discover, comme une demande.
- Un signal de production ne compte comme appris que s'il revient en Discover sous forme d'expression de besoin, au même niveau d'exigence qu'une demande neuve. Une mise à jour de statut ne suffit pas.
- Pas de sortie par défaut : une analyse incomplète laisse l'incrément en observation.

Chaque phase remet une spec à la suivante

Module 1 · slide « Chaque phase remet une spec à la suivante »

Phase	Ce qu'elle remet	Chez Vélora
Discover	Business Intent Spec	le problème, les utilisateurs, un critère de succès chiffré, son responsable
Design	Solution Design Spec	le contrat d'API de l'offre en v1, les critères d'acceptation, des NFR chiffrées
Develop	Code + Test Specs	le code, et un test par critère d'acceptation
Deploy	Runtime Specs	le runbook, le flag, le retour arrière, le coût IA relevé
Deepen	Opportunity Spec	continuer, ajuster ou arrêter, au regard du critère de départ
Drop	Retirement Spec	retirer la vente au comptoir : dépendances et impacts

Entre deux phases, on ne se passe pas un ticket ou un mail : on se passe une sortie contractualisée, relue et signée. C'est le principe du Spec-Driven Development :

> Le contexte dit à l'IA ce qu'elle doit savoir ; la spec, ce qu'elle a le droit et le devoir de faire.

Le contexte partagé (le Context Nexus, module 3) circule autrement : toutes les phases le lisent, tout le temps, et la gate ne le contrôle pas.

La grille des six phases

Module 1 · slide « Zoom sur une phase : Develop »

Chaque phase suit la même grille. La slide détaille Develop ; voici les six, en deux tableaux.

Ce qui entre, ce qui sort

Phase	But	Entrée	Sortie	Gate de sortie
Discover	Cadrer un besoin qui vaut d'être traité	Un brief métier, ou un retour de Deepen ou de Drop	Business Intent Spec : intention, objectifs et métriques, solutions existantes repérées	G1 Ready to Spec

Phase	But	Entrée	Sortie	Gate de sortie
Design	Transformer l'intention en contrat de solution	Business Intent Spec	Solution Design Spec : spec fonctionnelle, stories, critères d'acceptation, contrat d'API, versions de design figées, exigences non fonctionnelles	G2 Ready to Build
Develop	Produire du code conforme à la spec	Solution Design Spec	Code + Test Specs : le code, ses tests, la doc, la spec mise à jour de ce que l'implémentation a appris	G3 Ready to Release
Deploy	Livrer de façon traçable et observable	Code + Test Specs	Runtime Specs : conditions d'exploitation, observabilité technique et métier, alertes et escalade, retour arrière	G4 Released and Observable
Deepen	Mesurer la valeur réelle et conclure	Runtime Specs	Opportunity Spec, ou une recommandation de retrait	G5 Opportunity or Retire
Drop	Retirer proprement	Recommandation de retrait	Retirement Spec : le périmètre du retrait	G6 Retirement Scoped

Qui fait quoi

Phase	Ce que fait l'IA	Ce que fait l'humain	Jamais délégué	Responsable
Discover	Recherche le contexte ; rédige un brouillon de l'expression de besoin ; vérifie les données disponibles et l'adéquation à l'architecture	Formule l'intention	Si ça vaut la peine, et sa priorité	Produit, avec l'UX
Design	Vérifie ce qui est réutilisable ; interroge la spec par discipline ; propose des options, des prototypes, des brouillons de stories	Choisit parmi les options	L'architecture et les critères d'acceptation	PO, UX, tech lead et architecte, ensemble

Phase	Ce que fait l'IA	Ce que fait l'humain	Jamais délégué	Responsable
Develop	Contrôles déterministes (lint, analyses, tests) ; génère code, tests et doc ; aide à la remédiation et au refactoring	Relit, signe, tient le jugement d'ingénierie	L'approbation en périmètre sensible et en comité de changement	Les développeurs, la QA pour le rapport de test
Deploy	Capture les traces de déploiement et de flags ; orchestre la release et la vérifie après coup	Décide de la release	L'activation en périmètre critique, le déploiement en périmètre sensible	La QA et les équipes plateforme
Deepen	Observe la production ; détecte les régressions ; assemble les preuves d'usage, de performance et de coût	Conclut : continuer, faire évoluer ou retirer	L'arbitrage valeur / coût / risque	Les équipes data analysent, le produit conclut
Drop	Cartographie dépendances, données et intégrations ; inventorie les utilisateurs touchés ; extrait ce qui se réutilise	Communique auprès des utilisateurs touchés	La décision de retrait	Le produit, avec les métiers

L'unité de travail devient la spec

Module 1 · slide « L'unité de travail devient la spec »

Aujourd'hui, souvent	Dans le nouveau cycle
Un ticket décrit une tâche	Une spec décrit l'intention, les critères et les contrats
Le développeur écrit le code	Le développeur valide et tient le jugement d'ingénierie
La revue arrive à la fin	La preuve est demandée à chaque passage de phase
Les prompts restent dans les têtes	Prompts, contexte et règles sont versionnés avec le code

Le travail se déplace vers le contexte, les specs et les décisions ; la vitesse de frappe passe au second plan. Règle d'or : l'agent rédige, un humain relit et signe. Une spec que personne n'a relue n'est pas une spec.

Le long du flux, quelques convictions :

- **Idea** : une seule porte d'entrée, l'expression de besoin. Pas d'expression de besoin, pas de spec.
- **Shape et spec** : l'intention vit dans la spec, pas dans le code. La spec a un responsable nommé, elle épingle les versions de ce qu'elle utilise, et elle est mise à jour quand le code diverge ; ce que l'implémentation invente est signalé.

- **Code** : le déterminisme d'abord (hooks, linters), les agents ensuite. Les règles des agents reflètent celles de la CI.
- **Build et review** : l'approbation suit la criticité. Pas d'approbation par un agent en périmètre sensible ; chaque agent agit sous sa propre identité.
- **Deploy** : Release ≠ Activation. Chaque déploiement et chaque bascule de flag sont tracés.
- **Observe** : le contexte est épinglé par tâche, et on mesure la dérive.
- **La boucle** : chaque incident revient comme un changement court qui enrichit le contexte.

Qui tient le verdict ?

Module 1 · slide « Qui tient le verdict ? »

Un niveau de délégation dit qui tient le verdict, quelle que soit la quantité d'IA utilisée.

Niveau	Qui produit	Qui tient le verdict	Ce que l'humain voit
Human	Une personne	Une personne	Tout
AI-assisted	L'IA propose	Une personne	Chaque sortie, avant qu'elle compte
AI-executed	L'IA	Un contrôle automatique	Un échantillon, et chaque échec
AI-autonomous	L'IA	Un contrôle automatique	Les exceptions seulement

- **Un niveau se donne à une activité**, pas à une équipe ni à un outil. Un développeur qui génère presque tout son code mais décide du merge : AI-assisted. Une CI qui bloque un merge d'elle-même : AI-executed.
- **AI-autonomous** est réservé aux actions déterministes et réversibles (tests, analyse, capture de traces) : il ne produit jamais d'intention et ne décide jamais.
- **L'attribution suit le verdict** : au-delà d'AI-assisted, ce que l'IA produit est attribué à l'exécution elle-même. La trace, l'identité de l'agent et le coût doivent donc être en place avant.

La rampe : l'autonomie se gagne

Module 1 · slide « La rampe : l'autonomie se gagne »

- **Monter d'un cran** demande une preuve dont le seuil est fixé avant la période de mesure, jamais choisi après coup. Pas de gate, pas de montée au-delà d'AI-assisted.
- **Redescendre est automatique** : si la qualité baisse (les défauts qui passent la gate augmentent), l'activité revient au niveau inférieur, sans décision.
- **Le calendrier ne fait rien avancer.** Pendant le pilote, AI-assisted partout et un humain sur chaque tâche : c'est un plafond, et personne n'est tenu de l'atteindre.
- **Qui fait avancer une étape** : proposé sur preuve mesurée, enregistré, puis ratifié par une instance de gouvernance encore en cours de définition.

Trois plafonds, quelle que soit la preuve :

1. **Le jamais délégué** reste humain, une activité par phase : si ça vaut la peine et sa priorité (Discover), l'architecture et les critères d'acceptation (Design), l'approbation en périmètre sensible (Develop),

l'activation en périmètre critique (Deploy), l'arbitrage valeur / coût / risque (Deepen), la décision de retrait (Drop). Certaines activités le restent pour des raisons réglementaires, d'autres parce qu'elles demandent un jugement : une gate ne peut pas répondre de ses actes.

2. **Le périmètre sensible** ne dépasse pas AI-assisted.
3. **Pas d'autonomie sans trace** : chaque action tracée, attribuée, son coût connu.

Point de vigilance : quand on approuve presque tout, la relecture peut devenir un tampon. L'étape glisse alors vers de l'AI-executed que personne n'a décidé ni mesuré.

> Demandez « qui tient le verdict ? », pas « combien d'IA ? ».

Les principes du programme

Modules 0 et 1 · pour aller plus loin

Les règles qui guident la transformation, et qu'on retrouve dans chaque module :

1. **Le résultat avant la production** : on prouve l'adoption par un événement réel, pas par un taux de complétion.
2. **Ce qui n'est pas écrit n'existe pas** : ce qui n'est pas dans le cadre d'usage n'est pas une règle.
3. **Le contrôle ne prend jamais de retard sur l'adoption** : pas de nouvelle vague tant que l'écart se creuse.
4. **Chaque exception a un nom** : une exception anonyme devient le nouveau standard.
5. **Pas de capacité sans compteur** : aucun agent en service sans compteur, quota et responsable.
6. **Qui maîtrise écrit** : ceux qui savent écrivent les standards, et les prouvent par écrit.
7. **Réparer le processus d'abord** : on nomme l'écart avant d'accepter le cas d'usage.
8. **Rien ne compte tant que ce n'est pas mesuré** : une gate demande une preuve ; une déclaration ne suffit pas.
9. **L'accès avant les compétences** : on outille d'abord, on forme ensuite.
10. **L'adoption est une distribution** : une équipe en difficulté n'est jamais compensée par la moyenne.
11. **La télémétrie d'abord** : les systèmes avant les sondages.
12. **Les métriques sont visibles** : pas de donnée collectée sans restitution.
13. **Le contexte ne se présume jamais** : les conventions tacites échouent en silence ; les nôtres échouent bruyamment.

Encore à caler

Module 1 · pendant le pilote

- **Le curseur de délégation** : aucune règle ne le place encore, activité par activité.
- **La place du niveau AI-autonomous** : faut-il quatre niveaux, ou trois ?
- **L'identité des agents en revue** : sous quelle identité un agent relit et approuve.
- **L'érosion de la vigilance** : comment détecter qu'une relecture est devenue un tampon.

FICHE MODULE 2

Les quality gates

Où le travail s'arrête, sur quelles preuves, et qui décide : ce qu'est une gate, les deux niveaux de gates, les six gates de la roue, ce qui change au quotidien, les garde-fous et la calibration.

Une gate vérifie des preuves, ce n'est pas une réunion

Module 2 · slide « Une gate vérifie des preuves, ce n'est pas une réunion »

Une gate porte des assertions vérifiables, chacune avec un seuil, que la machine évalue sur des artefacts qui existent déjà. Elle n'ajoute ni réunion ni document.

1. **L'équipe demande**, sur ce qu'elle a déjà produit : spec, code, tests, runbook.
2. **La machine vérifie** des critères chiffrés, sans jugement : c'est le gate-check, que l'équipe peut lancer elle-même avant son créneau.
3. **Un humain décide** GO ou NO-GO, en quelques minutes, sur ce que la machine ne voit pas.
4. **C'est tracé** : quels critères, quelle version de la gate, qui a décidé (humain, ou agent avec son identité), combien ça a coûté. Une décision tracée se rejoue.

> Une gate se passe sur preuve, jamais sur déclaration.

Une gate n'est pas un rituel : un rituel sert à s'aligner, juger, arbitrer ; une gate vérifie un contrat. Seule la sortie d'une phase passe la gate : le contexte partagé reste hors de son contrôle.

Deux niveaux de gates

Module 2 · slide « Deux niveaux de gates »

	Gates de phase	Gates de tâche
Où	À chaque passage de phase	Entre deux étapes d'une tâche confiée à un agent
Ce qu'elles protègent	L'incrément livré	L'exécution et son contexte
Cadence	Une fois par incrément	À chaque exécution, plusieurs fois par jour
Qui tranche	Un humain nommé	Un contrôle automatique, avec un checkpoint humain sur la plupart des étapes
En cas d'échec	L'incrément revient dans la phase	L'étape est rejouée

Une gate de tâche contrôle en deux couches : d'abord des règles déterministes (lint, schéma, tests, budget), puis la grille d'un agent critique (conformité, structure, données, risques). Les quatre checkpoints humains :

- **Approuver** : l'intention est-elle bien posée ?

- **Valider** : l'analyse tient-elle ?
- **Choisir** : quelle option retenir ?
- **Signer** : la livraison est-elle acceptée ?

Règles des gates de tâche :

- une gate qui échoue renvoie à l'étape, jamais d'avertissement qui laisse passer ;
- une gate qui ne sait pas trancher remonte à l'humain au lieu de deviner ;
- elle peut renvoyer une action humaine : « allez voir l'équipe d'architecture sur tel point » ;
- **budget de tokens dépassé = NO-GO**, au même titre qu'un test rouge : qualité et coût se lisent ensemble ;
- un contexte périmé (« stale ») fait échouer la gate.

Les gates de tâche rendent les gates de phase tenables : sans elles, tout s'accumule au dernier passage.

Les six gates

Module 2 · slide « Quatre gates à connaître »

Gate	Ce qu'on montre	Mode	Verdict tenu par	Verdict
G1 Ready to Spec	L'expression de besoin en 5 champs, un critère de succès chiffré et son responsable, les données disponibles, l'adéquation à l'architecture	Contrôle automatique, puis humain	Le produit	Prêt à spécifier
G2 Ready to Build	La spec relue, des critères évaluables, des NFR chiffrées, le contrat v1 publié, les versions figées	Contrôle automatique, puis humain sur l'intention de solution	PO, UX, tech lead, architecte	Definition of Ready
G3 Ready to Release	Tests et scans au vert au seuil convenu, la spec à jour, rien de stale, la trace de revue, le plan d'activation des flags	Automatique d'abord ; humain obligatoire en périmètre sensible	Les développeurs et la QA	Definition of Done
G4 Released and Observable	Le journal des déploiements et des flags, les KPI et alertes instrumentés, le runbook, le retour arrière, le coût IA attribué	Automatique ; signature humaine en périmètre sensible	La QA et les équipes plateforme	Livré et observable

Gate	Ce qu'on montre	Mode	Verdict tenu par	Verdict
G5 Opportunity or Retire	L'usage, la performance face à la promesse, les coûts (IA compris), les incidents	Humain, sur des preuves assemblées automatiquement	Les équipes data, puis le produit	Continuer, repartir en Discover, ou retirer
G6 Retirement Scoped	La carte des impacts (pour chaque élément : migrer, archiver ou supprimer), les utilisateurs touchés, ce qu'on garde	Humain	Le produit, avec les métiers	Le retrait repart en Discover

G2 et G3 donnent la même Definition of Ready et la même Definition of Done à toutes les équipes. Les NFR sont chiffrées, sans adjectif : « rapide » n'est pas un critère, « p95 sous 300 ms » en est un.

La machine vérifie, l'humain juge

Module 2 · slide « La machine vérifie, l'humain juge »

G2 Ready to Build

La machine vérifie	L'humain juge
Chaque story a des critères d'acceptation écrits comme des assertions qu'un agent peut évaluer	Les exigences non fonctionnelles sont chiffrées (performance, disponibilité, accessibilité, rétention)
Les versions des composants, maquettes et tokens référencés sont figées	La Definition of Ready est atteinte
Le contrat d'API est publié et valide	L'intention de solution est la bonne
La spec passe la grille : chaque question des disciplines (front, back, architecture, sécurité, data) est résolue dans la spec ou devient une action nommée	

NO-GO : retour en Design, avec les questions restées ouvertes et, pour chacune, la personne à aller voir.

G3 Ready to Release

- Les contrôles déterministes sont au vert, aux seuils convenus (couverture, mutation).
- Les failles de sécurité sont résolues, ou acceptées avec un responsable et une échéance. On passe ou on revient, jamais « on avertit et on merge ».
- L'implémentation est conforme à la spec ; tout écart, tout composant inventé est signalé.
- La revue nomme son relecteur et, si c'est un agent, l'identité sous laquelle il agit.
- La criticité est notée, et le circuit d'approbation y correspond.
- Un plan d'activation des flags existe : ce qui est livré, ce qui est activé, la condition de retour arrière.

NO-GO : retour en Develop, avec le critère en échec nommé.

Ce que la machine ne voit pas : la spec dit-elle la bonne chose ? Chaque critère a-t-il son test ? Le code respecte-t-il le contrat ? Qui signe, sous quelle identité ? Combien ça a coûté ? Le temps humain va là : on lit, on ne tamponne pas.

Ce qui change au quotidien

Module 2 · slide « Ce qui change au quotidien »

- **Un NO-GO nomme le critère**, et si besoin la personne à aller voir. On corrige, on revérifie, on redemande : ce n'est pas une sanction.
- **Release ≠ Activation** : déployer et activer sont deux décisions humaines, tracées, avec retour arrière. G4 se déclenche au déploiement, puis à chaque bascule de flag (voir le journal ci-dessous).
- **Une exception a un nom** : un responsable, une raison, une date de fin, et une trace. Sinon, elle devient la règle.
- **Pas de nouveau rituel** : les gates nourrissent les revues qui existent déjà (la revue produit hebdomadaire pour G1 et G5, la revue d'architecture pour G2 et G3, la revue mensuelle pour G4 et G6). On y ajoute un seul point, une fois par mois : la calibration.

Le journal des flags : une ligne par bascule.

```text
| AAAA-MM-JJThh:mm | nom-du-flag | on | prénom.nom | motif |
```

Une bascule sans ligne de journal est un NO-GO. Le retour arrière, c'est une bascule « off », sa ligne de journal et un message à l'équipe.

> Un NO-GO qui nomme son critère vaut mieux qu'un GO par gentillesse.

Les huit garde-fous

Module 2 · pour aller plus loin

1. **Le déterminisme d'abord** : linter, schéma ou assertion de pipeline avant tout agent juge. L'agent juge intervient là où une règle ne peut pas trancher, et d'abord sur des changements peu critiques. Les règles des agents reflètent celles de la CI.
2. **Une gate humaine par défaut** : chaque tâche a une gate humaine dès le pilote. La confiance se gagne par l'observation ; l'équipe versionne chaque assouplissement, aucun ne se décrète.
3. **L'approbation suit la criticité** : l'équipe déclare la criticité et la note. En périmètre sensible ou en comité de changement, aucune approbation par un agent : un humain authentifié à chaque étape.
4. **Une gate renvoie, elle ne bloque pas en silence** : chaque reste à faire devient une action nommée, avec un interlocuteur nommé.
5. **Pas de gate sans compteur** : chaque gate a un compteur, un seuil et un responsable. Une gate que personne ne mesure n'est qu'un rituel au ton plus sévère.
6. **Preuve, puis décision, puis trace** : on enregistre les critères, la version de la gate, qui a décidé et le coût en tokens.

7. **Des gates calibrées, et une calibration observée** : taux de passage, taux d'échappement, cadence de revue.
8. **Des exceptions nommées, tracées, avec une date de fin** : réservées aux incidents critiques et aux écarts d'équipe justifiés, avec un responsable, une raison et une échéance.

La calibration

Module 2 · slide « Ce qui change au quotidien »

Chaque gate a une version et des mesures ; l'équipe les relit une fois par mois :

- le **taux de passage** au premier essai ;
- le **taux d'échappement** : ce qui a passé la gate et a cassé plus tard ;
- le **nombre d'allers-retours** avant le GO.

Ce qu'on en fait :

- une gate qui ne se déclenche jamais est retirée ;
- une gate qui se déclenche toujours est mal réglée : on la recalibre ;
- un taux d'échappement qui monte fait redescendre l'activité d'un niveau de délégation, automatiquement ;
- une étape qui tient durablement un très haut taux de passage (par exemple autour de 97 % sur trois mois) peut alléger sa vérification humaine, sur un seuil fixé à l'avance : les échecs restants maintiennent la surveillance.

Encore à caler : la gate de bout en bout

Module 2 · pendant le pilote

Chaque gate peut passer alors que l'ensemble dérive : aucune gate ne vérifie encore la qualité telle que le client la vit, sur plusieurs équipes. Deux pistes, aucune n'est retenue :

- une gate agrégée, au niveau du train de release ;
- des rituels de convergence, tranchés par un arbitre.

Dans les deux cas, les équipes devront d'abord harmoniser leurs rituels et leurs artefacts. Nous trancherons avec les équipes pilotes.

FICHE MODULE 3

Le Context Nexus

Ce que l'équipe écrit et entretient pour que les agents travaillent bien : les quatre registres, le découpage des ADR, les niveaux, les contrats et leurs versions, et quand on met tout ça à jour.

Ce que l'agent ne devine pas

Module 3 · slides « Avec ou sans contexte » et « Ce que l'agent ne devine pas »

Le **Context Nexus** réunit en un seul système ce que l'équipe sait, veut, exige et fait, lisible par les humains et consommable par les agents.

- **Un agent ne devine rien** : il travaille avec ce qu'on lui donne, rien de plus. Une convention non écrite lui reste invisible ; une nuance implicite devient une erreur en production.
- **La documentation devient une entrée du travail** : l'agent la lit avant de produire. Sa qualité borne celle de ce qui en sort.
- **Il ne s'installe pas, il s'applique** : ni outil ni format, le Nexus réorganise ce que l'équipe a déjà (wiki, ADR, runbooks, DoD) sans ajouter de couche de documentation.

> L'IA est bonne en général ; elle ne connaît pas nos décisions.

Les quatre registres

Module 3 · slide « Quatre registres »

Registre	La question	Ce qu'on y trouve	Signé par	Comment l'agent le lit
Knowledge	Ce qu'on sait	Glossaire métier, carte d'architecture, conventions, anti-patterns, contraintes réglementaires, **Decision Contexts**	Tech lead	Cherché à la demande ; les conventions universelles sont toujours chargées
Purpose	Ce qu'on veut	Specs et user stories, critères d'acceptance, **Decision Directives**, objectifs, hypothèses produit	Produit	Injecté pour la tâche en cours ; les directives universelles sont toujours chargées
Contracts	Ce qu'on exige	Des assertions **vérifiables, avec un seuil**, évaluables sans jugement humain : DoD, sécurité, accessibilité, performance, contrats d'API, qualité des données	Tech lead, avec UX ou Data selon le domaine	Toujours chargés ; vérifiés par des outils (tests, contrôle d'accessibilité, validation de schéma)

Registre	La question	Ce qu'on y trouve	Signé par	Comment l'agent le lit
Process	Ce qu'on fait	Runbooks, procédures de déploiement et de retour arrière, playbooks d'incident	L'équipe	Exécuté : l'agent ne lit pas un runbook, il le déroule

Les signataires sont des rôles, pas des personnes : si quelqu'un change de poste, le registre garde un responsable.

« On vise une bonne accessibilité » ne se vérifie pas. « Toutes les interfaces respectent le niveau WCAG AA » est un contrat : il se vérifie, avec un seuil.

La règle d'or

Module 3 · slide « Quatre registres »

Purpose et Contracts restent humains, à toutes les phases : l'agent peut proposer, observer, signaler ; il ne formule jamais l'intention et ne fixe jamais les exigences. Cette règle rend le reste négociable.

Knowledge et Process passent aux agents par étapes :

1. l'humain écrit ;
2. l'agent propose, l'humain valide ;
3. l'agent entretient sous supervision, et signale les dérives.

Découper une ADR

Module 3 · slide « Une décision d'architecture (ADR), deux artefacts »

L'ADR classique mêle le pourquoi et la règle dans un seul document. Dans le Nexus, l'équipe découpe obligatoirement toute ADR acceptée en deux artefacts, parce qu'ils ont deux lecteurs différents :

	Decision Context	Decision Directive
Registre	Knowledge	Purpose
Contenu	La situation qui a rendu la décision nécessaire, les options évaluées, pourquoi on a écarté les autres, les conséquences observées	La règle d'exécution, courte, formulée en positif, directement applicable
Nature	De la mémoire, pas une directive	Une instruction, pas une explication
Lu par	L'humain qui veut comprendre (six mois plus tard, une nouvelle recrue), l'agent à la demande	L'agent, à chaque tâche du périmètre

La directive renvoie à son contexte et au contrat qui la vérifie : les critères que contrôle la gate vivent dans Contracts.

Exemple, chez Vélora : l'équipe Notifications décide que « les messages clients passent par VelNotif, pas par un prestataire d'envoi externe ».

```text
Decision Context (Knowledge)
  Situation : trois services envoyaient déjà des messages, chacun avec son prestataire.
  Options : garder un prestataire par service ; un service interne commun.
  Écarté : un prestataire par service (pas de plage « ne pas déranger » commune,
  doublons entre services, consentements gérés trois fois).
  Conséquences observées : à compléter après 30 jours en production.

Decision Directive (Purpose)
  Tout message client passe par l'API VelNotif.
  Contexte : knowledge/decision-contexts/velnotif.md
  Vérifié par : contracts/velnotif-api.md (aucun appel direct à un prestataire d'envoi)
```

Le niveau dépend de la portée de la décision : une décision d'équipe reste au niveau de l'équipe ; une décision qui vaut pour tout le groupe devient une directive universelle, au niveau de l'organisation.

Une décision ultérieure remplace explicitement l'ancienne (`supersedes`) et garde le lien vers elle.

La grille registres × niveaux

Module 3 · slide « Trois niveaux pour chaque registre »

Les registres disent ce qu'est une information ; les niveaux disent jusqu'où elle s'applique. Le Nexus est une grille :

	Knowledge	Purpose	Contracts	Process
Organisation	conventions universelles, anti-patterns	directives universelles	sécurité, accessibilité, performance de référence	runbooks partagés
Programme (facultatif)	glossaire commun, architecture du domaine	—	contraintes d'intégration entre équipes	déploiement commun
Équipe	connaissance métier, Decision Contexts	specs, directives de l'équipe	contrats de l'équipe, DoD	runbooks de l'équipe

- **Chaque niveau hérite** du niveau au-dessus et peut l'étendre, jamais le remplacer : ce qui est vrai en haut reste vrai en bas.
- Le niveau programme est facultatif et sert dès que deux équipes ou plus partagent un contexte. Il peut y en avoir plusieurs.
- Les specs ne vivent qu'au niveau de l'équipe.
- Chaque case a son responsable : au niveau organisation, l'architecture, la sécurité et la qualité, la plateforme ; au niveau programme, l'architecte ou le responsable technique du domaine ; au niveau équipe, le tech lead, le produit et l'équipe.

Étendre, déroger, contredire

Module 3 · slide « Trois niveaux pour chaque registre »

Cas	Ce que c'est	Ce qui se passe
Étendre	L'équipe ajoute ce que le niveau au-dessus ne couvre pas	Toujours autorisé
Déroger	L'équipe déclare une exception à une règle précise du niveau au-dessus	Prise en compte seulement si le responsable du niveau au-dessus l'a approuvée ; sinon, exclue du contexte donné à l'agent
Contredire en silence	L'équipe écrit l'inverse d'une règle du dessus, sans le déclarer	Bloqué et signalé au responsable

Une exception porte deux mentions : à quoi elle déroge, et qui l'a approuvée.

```yaml
register: contracts
level: équipe
exception-to: org/contracts/performance.md
exception-approved-by: responsable qualité de l'organisation
```

La dérogation silencieuse devient impossible : c'est soit une extension, soit une exception approuvée, soit un conflit bloqué.

Contrats, versions et épinglage

Module 3 · slide « Des contrats d'API publiés tôt évitent des réunions »

Un contrat entre équipes remplace une partie des réunions d'alignement : ce qu'une équipe promet aux autres est écrit, versionné et vérifiable.

- **Publier tôt une v1 courte** : les équipes qui en dépendent peuvent commencer. Un contrat parfait gardé pour soi ne sert à personne.
- **L'équipe numérote chaque version** ; les équipes consommatrices épinglent la version sur laquelle elles travaillent.
- **Un changement majeur** passe les consommateurs en **« stale »** : ce n'est pas une panne, c'est le signal qu'il faut se parler.
- **La spec avant le code** : relire le changement, mettre à jour la spec, puis les tâches, le code et les tests, épingler la nouvelle version, republier.

Les statuts d'un artefact : brouillon → actif → stale (plus validé depuis trop longtemps, ou dépendance changée) → de nouveau actif, ou déprécié, ou remplacé (le lien est gardé). Un artefact stale reste en place, signalé. Une gate qui s'appuie sur un contexte stale n'affirme rien : d'où le critère « rien de stale » de G3.

Contribuer sur événement

Module 3 · slide « Le Nexus se met à jour à des moments précis »

Toute base de connaissance meurt de la même façon : quand contribuer devient facultatif. Dans le Nexus, des événements que l'équipe vit déjà déclenchent chaque contribution.

Quand…	…on met à jour
une ADR est acceptée	on la découpe : Decision Context en Knowledge, Decision Directive en Purpose
un ticket est créé	on vérifie que la spec est structurée et reliée à ses contrats
une MR est mergée	les directives qu'elle touche
une gate échoue	le responsable tranche : seuil mal calibré, ou régression
un incident est résolu	le runbook, et le motif de fragilité dans Knowledge
une revue de sprint	on range les specs terminées
une revue d'architecture	on revalide les Decision Contexts existants

Les agents peuvent signaler un écart (« ce runbook ne correspond plus à ce que j'ai observé ») ; le responsable humain valide.

Plutôt que deviner, on écrit « à confirmer ». Chaque information dit d'où elle vient.

Comment l'agent reçoit le contexte

Module 3 · pour aller plus loin

Avant chaque tâche, l'outillage assemble un **task brief** en parcourant la grille de l'organisation jusqu'à l'équipe :

- **toujours chargé** (system prompt) : les règles et contrats qui s'appliquent partout ;
- **injecté pour la tâche** : la spec en cours et les directives qui la concernent ;
- **cherché à la demande** : la connaissance (glossaire, Decision Contexts, architecture) ;
- **exécuté** : les runbooks et les outils de vérification, sous forme de skills.

La façon dont l'agent lit un registre dicte la façon de l'écrire : une directive courte et sans nuance, un Decision Context complet et daté.

Démarrer

Module 3 · slide « Trois niveaux pour chaque registre »

- **Deux niveaux** suffisent pour commencer : organisation et équipe.
- **Les Contracts d'abord** : les plus rapides à rendre vérifiables, et le point où les équipes se cassent entre elles.
- Des **fichiers Markdown versionnés dans le dépôt** suffisent : 5 à 10 règles absolues de l'organisation, les contrats de l'équipe, un en-tête par fichier (registre, niveau, responsable, statut, version).
- Pendant les premières semaines, les humains écrivent tout ; les agents proposent ensuite.

Encore à caler

Module 3 · pendant le pilote

- **Qui tient le cycle de vie du Nexus de bout en bout** : la répartition par registre et par niveau est une proposition, pas encore une décision.
- **Le Context Engineer** (module 4) et les signataires par registre : comment les deux se combinent dans l'équipe.

FICHE MODULE 4

Équipe et nouveaux rôles

Qui porte quelle casquette dans l'équipe, avec quelle charge, et quels rôles arrivent autour d'elle : de quoi nommer les casquettes et repérer les surcharges avant que l'outillage n'arrive.

Même équipe, nouvelles casquettes

Module 4 · slide « Même équipe, nouvelles casquettes »

- **La forme de l'équipe ne change pas** : pas de réorganisation, pas de recrutement pour le pilote.
- Les nouveaux rôles sont des casquettes, pas des postes : des personnes déjà là les portent, en plus ou à la place d'une partie de leur travail actuel.
- On entre dans le pilote en équipe entière : tous les rôles produit ensemble. Une équipe à moitié embarquée garde deux façons de travailler.
- **La responsabilité suit le rôle** : si quelqu'un change de poste, la casquette passe à quelqu'un d'autre.

Métier	Ce qui change
PO, BA	Écrire des specs qu'une machine peut vérifier : critères d'acceptation, cas limites, invariants. Le PO est souvent le Specification Engineer naturel.
Développeur	Écrire moins, valider davantage et exercer son jugement d'ingénierie : relire, signer, décider.
Tech lead	Moins de « oui / non » au fil de l'eau, davantage de contexte à tenir et de décisions à porter.
Architecte	Faire converger les solutions des équipes vers une architecture commune ; co-responsable de G2.
QA	Évaluer ce que produisent les agents : jeux de données de référence, seuils, non-régression.
UX	Enrichir le design system que les agents réutilisent, plutôt que produire chaque maquette.

> Personne ne perd son métier : chacun ajoute une casquette, et en confie une partie à l'agent.

Les quatre casquettes de l'équipe

Module 4 · slide « Quatre casquettes dans l'équipe »

Casquette	Mission	Ce qu'elle tient	Porteur typique	Charge indicative
Context Engineer	Tenir le contexte que lit l'agent, pour qu'une demande soit exécutable sans tout redécouvrir	Glossaire, ADR, modèles de données, contrats d'interface, règles métier ; un contexte sourcé, daté, réutilisable, sans contradiction (Definition of Ready du contexte)	BA senior, tech lead	~0,25 ETP
Specification Engineer	Transformer l'intention produit en spec qu'une machine peut vérifier	Specs, critères d'acceptation, invariants, cas limites ; rien ne part en génération avec une question ouverte (Definition of Ready de la génération)	PO, BA, architecte	~0,35 ETP
Agentic Workflow Orchestrator	Organiser le travail des agents	Découpage des tâches, enchaînement des agents, points de relecture, budget de tokens (Definition of Done de la génération)	Tech lead, staff engineer	~0,3 ETP, tournant sur 2 ou 3 personnes
Decision Guarantor	Porter, en son nom, la responsabilité de ce qui part en production	La décision de mise en production ; la liste des décisions de l'équipe qu'on ne délègue jamais à un agent	Tech lead, expert du domaine	~0,15 ETP, nommé, jamais tournant ni délégué

Les charges sont des ordres de grandeur, encore en discussion : chaque équipe les ajuste à sa taille et à son contexte.

L'Evaluation Engineer, depuis un pool

Module 4 · slide « Quatre casquettes dans l'équipe »

Obligatoire dès le pilote, mais partagé entre plusieurs équipes (environ 0,25 ETP par équipe). Il prouve qu'une sortie tient la Definition of Done sur la durée, pas sur un seul essai : jeux de données de référence, banc d'évaluation, seuils, suites de non-régression. Il propose aussi, preuves à l'appui, de faire avancer une activité sur la rampe de délégation (module 1).

Porteurs typiques : QA ou SDET, développeur fonctionnel senior, chercheur UX expérimenté.

Le premier goulot à repérer

Module 4 · slide « Le premier goulot à repérer »

Le tech lead est le porteur typique de trois des quatre casquettes :

Casquette	Charge indicative
Context Engineer	~0,25 ETP
Agentic Workflow Orchestrator	~0,3 ETP
Decision Guarantor	~0,15 ETP
Total, en plus de son poste	**≈ 0,7 ETP**

Trois leviers :

1. **Répartir** : le contexte à un profil qui aime structurer (BA, UX, data) ; la spec au PO.
2. **Faire tourner** l'Orchestrator sur 2 ou 3 personnes.
3. **Garder stable** le Decision Guarantor : c'est la casquette qui demande le plus de continuité.

Quelques règles :

- le Decision Guarantor est une personne nommée ; il ne tourne pas et ne se délègue pas au fil de l'eau ;
- l'Orchestrator tourne, pour ne pas reposer sur une seule personne ;
- une personne peut porter plusieurs casquettes, tant que la charge reste tenable ;
- conseil : éviter qu'une seule personne instruise et décide seule ; la personne qui prépare une décision et celle qui la signe gagnent à être deux ;
- un profil manquant (pas de BA, pas de QA) est un écart à noter et à remonter, pas une faute.

Une surcharge révèle ce que la personne porte déjà aujourd'hui.

Autour de l'équipe : les rôles par vague

Module 4 · slide « Autour de l'équipe »

Plus on avance, moins il s'agit de créer un poste : le rôle prolonge souvent un rôle qui existe déjà.

Pilote

Rôle	Mission	Où	Porteur typique	Charge indicative
Les quatre casquettes	Voir plus haut	Dans l'équipe	—	—
Evaluation Engineer	Prouver la qualité des sorties sur la durée	Partagé entre équipes	QA, SDET, chercheur UX	~0,25 ETP par équipe

Passage à l'échelle

Rôle	Mission	Où	Porteur typique	Charge indicative
AgentOps	Tenir une plateforme d'agents stable, versionnée, réversible : connecteurs, passerelle et quotas, secrets, versions des modèles et des prompts, retour arrière	Partagé entre équipes	Ingénieur plateforme, SRE	1 à 2 ETP pour 5 équipes
Capability Librarian	Faire passer une bonne pratique locale (un skill, un prompt) dans la bibliothèque commune, et retirer ce qui est obsolète	Partagé entre équipes, avec les AI Champions	Knowledge manager, architecte de référentiel	0,5 à 1 ETP
Convergence Arbitrator	Ramener les solutions des équipes vers une architecture commune, repérer les doublons, décider d'arrêter	Gouvernance	Un collège de 4 à 6 architectes	~2 h par semaine

Industrialisation

Rôle	Mission	Où	Porteur typique	Charge indicative
FinOps IA	Rendre visibles et arbitrer les coûts d'exécution : budgets, coût par équipe et par fonctionnalité	Gouvernance	FinOps cloud, contrôle de gestion	~0,3 ETP
Trace & Audit Steward	Produire des traces exploitables et réutilisables, sans risque juridique : schéma, rétention, pseudonymisation	Gouvernance	Data engineer, DPO opérationnel	~0,5 ETP
Risque modèles et fournisseurs	Anticiper les ruptures : modèles, licences, réglementation, réversibilité ; feu vert avant tout nouveau modèle	Gouvernance	Risk officer, architecte d'entreprise, achats	0,3 à 0,5 ETP

Qui tient quoi aux gates

Module 4 · pour aller plus loin

Gate	Responsable	Casquettes impliquées
G1 Ready to Spec	Le produit	Context Engineer (le contexte de départ)

Gate	Responsable	Casquettes impliquées
G2 Ready to Build	PO, UX, tech lead, architecte, ensemble	Specification Engineer produit la spec ; Convergence Arbitrator tranche les divergences entre équipes
G3 Ready to Release	Les développeurs et la QA	Decision Guarantor signe ; Evaluation Engineer tient les seuils ; Orchestrator tient le découpage
G4 Released and Observable	La QA et les équipes plateforme	AgentOps ; plus tard, FinOps IA et Trace & Audit
G5 Opportunity or Retire	Les équipes data, puis le produit	Evaluation Engineer, Capability Librarian
G6 Retirement Scoped	Le produit, avec les métiers	—

Les AI Champions

Module 4 · slide « Autour de l'équipe »

Des relais dans les équipes, pas une équipe parallèle. Ils restent dans leur équipe et :

1. accélèrent l'adoption sur le terrain ;
2. traduisent la stratégie en cas d'usage concrets ;
3. font remonter les retours ;
4. partagent les pratiques entre pairs ;
5. reprennent le flambeau après le programme.

Ils travaillent avec le Capability Librarian pour faire circuler ce qui marche. Attention à leur charge : un Champion qui porte aussi une casquette cumule trois rôles.

Encore à caler

Module 4 · pendant le pilote

- **Le Context Engineer** : une casquette par équipe pendant le pilote ; transverse ensuite, comme l'architecture ou la sécurité ?
- **La convergence** : un collège d'architectes, un centre d'excellence, ou les AI Champions ?
- **Le Specification Engineer** : il produit la spec, il la valide, ou les deux ? Qui répond de sa qualité ?
- **Le Decision Guarantor** : comment il s'articule avec la gouvernance existante.
- **Casquettes et intitulés de poste** : la correspondance reste à établir.

FICHE MODULE 5

La toolkit

Avec quoi travaille une équipe, et sous quelles règles : les outils de chaque étape, Spec Kit commande par commande, le kit qu'une équipe reçoit, et la passerelle unique vers les modèles.

Un outil par étape

Module 5 · slide « Un outil par étape »

Trois choix de référence : Claude Code comme agent de code, Spec Kit pour le Spec-Driven Development, LiteLLM comme passerelle vers les modèles. Pour le reste, nous décrivons des catégories : la fonction compte plus que la marque.

Étape	Avec quoi	La conviction
Idea	Un assistant généraliste, le backlog connecté	Une seule porte d'entrée, l'expression de besoin. Pas d'expression de besoin, pas de spec.
Spec et shape	Spec Kit ; l'outil de design et le design system, lus par l'agent	L'intention vit dans la spec, pas dans le code. Tout est contexte, y compris le design system.
Code	Claude Code ; des hooks et des linters	Le déterminisme d'abord. Les règles de l'agent sont celles de la CI.
Build et review	La CI, des scanners de sécurité accessibles à l'agent, des agents relecteurs	L'approbation suit la criticité ; pas d'approbation par un agent en périmètre sensible.
Deploy	La CI/CD, les feature flags, la gestion des changements	Release ≠ Activation : chaque déploiement et chaque bascule de flag sont tracés.
Observe	Les logs, la supervision, les traces des agents	On épingle le contexte de chaque tâche et on mesure la dérive.
Partout	LiteLLM, la passerelle vers les modèles	Un seul chemin gouverné vers les modèles.

Spec Kit, commande par commande

Module 5 · slide « Un outil par étape »

Commande	Ce qu'elle produit	Où, sur la roue
(avant Spec Kit)	L'expression de besoin, rédigée avec l'agent, validée par un humain	Discover, jusqu'à G1
`constitution`	Les principes que toute spec respecte	Design

Commande	Ce qu'elle produit	Où, sur la roue
`specify`	La spec fonctionnelle : le quoi et le pourquoi, avec ses critères d'acceptation	Design
`clarify`	Les questions ouvertes, tranchées ; celles qui touchent une autre équipe deviennent une discussion avec elle	Design ; de nouveau en Develop si un contrat change
`plan`	Le plan technique, à partir de la stack et des contrats épinglés des autres équipes	Design
`tasks`	Le découpage en tâches	Design
`analyze`	Un rapport de cohérence entre spec, plan et tâches, à corriger avant de coder	Design, jusqu'à G2
`implement`	Le code et ses tests, tâche par tâche	Develop, jusqu'à G3
`converge`	Ce qui manque encore par rapport à la spec, ajouté en nouvelles tâches	Develop, après un changement

L'équipe n'invente pas la constitution : elle reprend les règles et contrats de l'organisation (module 3), chacun avec sa source. S'il manque un principe, elle le signale au lieu de l'ajouter de son côté.

Les couches de la plateforme

Module 5 · pour aller plus loin

Ce qui se trouve sous les outils de l'équipe, de l'IDE jusqu'à la mesure :

1. **Les agents de code** : le code se produit dans l'IDE, la qualité se décide dans la spec.
2. **L'outillage SDD** : la spec devient le contrat d'exécution de l'agent.
3. **La passerelle** : un seul point d'entrée vers les modèles, avec quotas, authentification et traçabilité.
4. **Le registre d'agents et les connecteurs MCP** : des agents et des skills versionnés, un accès standard aux outils.
5. **La fabrique de contexte** : conventions, ADR, runbooks et specs rendus lisibles par les agents (le Context Nexus).
6. **L'orchestration multi-agents** : plusieurs agents spécialisés sur une même tâche.
7. **Les versions de prompts et le FinOps** : prompts et règles versionnés comme du code, consommation par équipe et par cas d'usage.
8. **La CI/CD et les quality gates** : le code produit par l'IA tenu aux mêmes exigences, sans gate allégée.
9. **L'observabilité** : les métriques de delivery et d'usage de l'IA, ensemble.

L'ordre compte : le contexte et la passerelle avant l'orchestration. Un contexte métier structuré vaut plus qu'un orchestrateur de plus.

Le kit d'une équipe

Module 5 · slide « Dire, interdire, bloquer, prouver »

Ce que la plateforme fournit à une équipe qui entre dans le pilote :

1. **Un fichier d'instructions pour l'agent** : son périmètre ; ce que décident les humains (l'intention et les contrats) ; s'arrêter et signaler un écart avec un contrat plutôt que choisir une interprétation ; ne pas produire de code sur un contexte périmé ; un test par critère d'acceptation ; être économe en tokens.
2. **Des permissions versionnées** : ce que l'agent peut lancer (tests, lint, contrôles de gate) et ce qui lui est interdit (activer un flag).
3. **Un hook de garde** pour les 2 ou 3 règles qu'un prompt ne garantit pas : activer un flag, écrire hors du périmètre de l'équipe, modifier un fichier généré. Chaque blocage est expliqué et journalisé.
4. **Une CI qui rejoue les contrôles des gates** : lint, typage, tests et couverture, scans de sécurité et de dépendances ; le même contrôle que l'équipe peut lancer avant son créneau de gate.
5. **Spec Kit**, avec une constitution sourcée.
6. **Des scripts réservés aux humains, tracés** : activer un flag avec son motif (une ligne de journal), signer le code en périmètre sensible.
7. **L'accès au Context Nexus** : lecture de tout le contexte, écriture dans le seul dossier de l'équipe.

Dire, interdire, bloquer, prouver

Module 5 · slide « Dire, interdire, bloquer, prouver »

Une instruction ne suffit pas : un agent peut l'oublier. La même règle, « un agent n'active jamais un feature flag », tient sur quatre couches :

Couche	Ce qu'elle fait	Pour la règle du flag
L'instruction demande	Le fichier d'instructions de l'agent	« Tu ne bascules jamais un flag toi-même. »
La permission interdit	La liste autorisé / interdit	La commande d'activation est refusée à l'agent.
Le hook bloque	Une règle exécutée avant chaque action	Toute tentative, même par une modification de fichier, est bloquée et journalisée.
La gate prouve	La CI et le contrôle de gate	G4 vérifie que chaque flag actif a sa ligne de journal, signée par un humain.

Les règles de l'agent et celles de la CI sont symétriques : sinon, rien ne passe. Un garde-fou ne bloque jamais le travail par accident : s'il tombe en panne, il laisse passer, et la gate vérifie quand même.

Une seule porte vers les modèles

Module 5 · slide « Une seule porte vers les modèles »

- **Un seul point d'entrée** : chaque appel passe par la passerelle (LiteLLM). Sans elle, ni maîtrise des coûts, ni audit, ni gouvernance.

- **L'appel nomme une tâche et une classe de données**, pas un fournisseur : la passerelle route. Une règle qui dépend du bon choix de chaque personne qui code reste une convention ; la passerelle en fait un contrôle.
- **Une identité et un quota par agent et par équipe** : un jeton partagé entre plusieurs agents rend la consommation impossible à attribuer.
- **Un budget de tokens par exécution** : le dépasser est un NO-GO, lu avec le verdict qualité.
- **Chaque gate relève le coût.**

Classe de données	Modèles permis	Hébergement	Approbation
Publiques	Tout modèle admis	Tout fournisseur admis	Automatique
Internes	Modèles admis, sans entraînement sur nos données	Région ou environnement dédiés	Automatique
Confidentielles	Liste restreinte	Juridiction et environnement dédiés	Accord humain
Réglementées (paiement, données personnelles)	Liste restreinte et auditée	Fournisseur sous contrat, dans le périmètre	Humain authentifié, jamais un agent

Les modèles sont admis, revus et retirés selon des critères connus, et nous gardons la possibilité d'en changer, tâche par tâche. La passerelle est aussi un point unique de défaillance : en mode dégradé, les humains attendent les tokens.

> Lisez le coût avec la qualité, jamais seul.

L'économie des tokens

Module 5 · slide « Une seule porte vers les modèles »

Les réflexes de l'équipe

- **Une session neuve après chaque gate** : l'agent repart d'un contexte propre et du brief à jour, et la session coûte moins.
- **Lire un fichier précis** plutôt que tout le dépôt.
- **Suivre le budget** en direct : c'est le rôle de l'Orchestrator (module 4).

Ce qui fait la facture : le modèle appelé, la taille du contexte, le nombre d'itérations, les boucles et relances des agents, les sessions abandonnées.

Pour un manager : s'attendre à des frictions de budget au début ; lire le coût au niveau de l'équipe, jamais par personne, et toujours à côté de la qualité. Le coût IA par point de complexité livré figure parmi les indicateurs du module 6.

Encore à caler

Module 5 · pendant le pilote

- **Le routage par sensibilité** : les règles par classe de données ne sont pas encore toutes appliquées par la passerelle.
- **Qui admet un nouveau modèle**, et qui accorde une dérogation.
- **L'identité des agents en revue** : sous quelle identité un agent relit et approuve.
- **Le contexte d'une tâche** : l'épingler tâche par tâche, ou versionner le contexte global.

FICHE MODULE 6

Déploiement et mesure

Quand une équipe bascule, ce qu'elle vit mois par mois, et comment on saura si ça marche : le parcours, le calendrier type, les principes de mesure, les sept indicateurs et ce que l'équipe doit tenir propre pour qu'ils se calculent.

Le parcours : cadrer, piloter, passer à l'échelle

Module 6 · slide « Le parcours : cadrer, piloter, passer à l'échelle »

Étape	Ce qui se passe	La règle
Cadrage	On définit les indicateurs, on vérifie qu'on sait les calculer, on mesure le point de départ	Aucune délégation revendiquée : on mesure, on ne promet rien
Pilote	Quelques mois, des équipes entières, une collecte chaque semaine	AI-assisted partout, un humain sur chaque tâche
Passage à l'échelle	Par vagues ; chaque nouvelle équipe reçoit le même kit (module 5)	Une étape n'avance que sur preuve, avec un seuil fixé à l'avance

- **Aucune nouvelle vague ne s'ouvre tant que l'écart se creuse** : l'écart entre ce qu'on déploie et ce qu'on sait encadrer par les gates, la trace et la mesure. Le cadre ne court jamais derrière l'adoption.
- **On équipe d'abord, on forme ensuite** : une formation sur des outils que l'équipe n'a pas encore ne sert à rien.
- **Des équipes entières** : tous les rôles produit embarquent ensemble : PO, UX, architecture, développement, data, exploitation.
- **Un domaine en difficulté ne se compense pas par la moyenne** : on regarde chaque équipe, pas seulement le total.
- Le pilote se conclut, avec les équipes, par une rétrospective et une évaluation du passage à l'échelle.

Le mot « vague » désigne ici les vagues de déploiement. Les rôles arrivent eux aussi par vagues (module 4) : ce n'est pas la même chose.

Le calendrier type d'une équipe

Module 6 · slide « Le calendrier type d'une équipe »

Un calendrier en mois relatifs : votre programme fixe les dates.

Quand	Ce qui se passe	Ce que fait le manager
Avant M1	La mesure de référence, avec les mêmes indicateurs qu'après, dès que ses prérequis sont là (indicateurs définis, données récupérables, champs de l'outil de suivi en place) ; la première enquête d'expérience	Mettre l'outil de suivi au propre ; faire remplir l'enquête

Quand	Ce qui se passe	Ce que fait le manager
M1	L'équipe entière est inscrite, les quatre casquettes sont nommées	Nommer les casquettes, protéger leur temps
M1	Le poste est équipé (Claude Code, Spec Kit, accès à LiteLLM) ; chacun se positionne et prend en main les outils	Vérifier que chacun a ses accès avant de former
Fin M1	Le hackathon : le cycle complet sur un cas fictif	Libérer l'équipe pour la journée
Fin M1	Le premier incrément réel passe par la roue, de G1 à G3	Choisir un premier incrément modeste ; protéger le temps des gates
M2	Les parcours par métier ; le Nexus est amorcé, par les Contracts	Libérer le temps des parcours ; nommer qui signe chaque registre
M3	L'économie des tokens ; G4 en production ; la première calibration mensuelle des gates	Lire les premiers chiffres avec l'équipe, sans conclure trop tôt
Fin M4	La rétrospective et l'évaluation du passage à l'échelle	Faire remonter ce qui marche et ce qui coince

La montée en compétence

Module 6 · slide « Le calendrier type d'une équipe »

Étape	Pour qui	Ce qu'on y fait
Auto-positionnement	Tout le monde	Un parcours pratique, en autonomie, qui situe chacun et l'oriente
Prise en main des outils	Tout le monde	Les outils du poste, sur des cas d'usage de son métier
Hackathon	L'équipe entière	Le cycle complet, de l'idée à la mise en production, sur un cas fictif
Parcours par métier	Produit, UX, développement, data, exploitation	Ce qui change dans son métier ; les casquettes
Économie des tokens	Tout le monde, plus ou moins long selon le métier	Ce qui fait la facture, et les réflexes pour la tenir
Relais	Les AI Champions	Ils relaient les formations, font remonter les retours, puis prennent le relais

Rien ne forme mieux que la pratique. On s'appuie sur les communautés IA qui existent déjà plutôt que d'en créer une nouvelle.

Mesurer la valeur, pas l'adoption

Module 6 · slide « Mesurer la valeur, pas l'adoption »

1. **Pas de valeur sans référence** : on mesure le point de départ de chaque équipe avant d'outiller, avec les mêmes indicateurs qu'après, dès que les prérequis sont là : les indicateurs définis, les données récupérables, les champs de l'outil de suivi en place. Une référence reconstruite après coup, de mémoire, ne vaut rien.
2. **La valeur, pas l'adoption** : le taux d'usage ou la part de code écrite par l'IA montrent que l'outil sert ; ils ne disent rien de la valeur créée.
3. **Pas de vitesse sans qualité** : chaque indicateur de vitesse a son indicateur de qualité. Mesurer la vitesse seule pousse à optimiser le débit et dégrade la qualité plus loin.
4. **Le coût se lit avec la qualité** : productivité et consommation se mesurent ensemble.
5. **L'équipe, jamais la personne** : tout est agrégé par équipe ou par produit ; l'enquête d'expérience est anonyme, lue par rôle et par équipe.

On fixe les seuils avant la période mesurée, jamais après pour justifier une conclusion.

> Un indicateur de qualité ne prouve pas un gain : il réfute un gain apparent.

Les sept indicateurs

Module 6 · slide « Sept indicateurs, quatre dimensions »

Un seul jeu d'indicateurs pour toutes les équipes du pilote, pondéré par la complexité, sur un standard commun de l'outil de suivi.

Dimension	Indicateur	Comment, en une phrase
Vitesse	Lead time	De la création de l'epic à sa résolution ; la médiane du trimestre
Vitesse	Cycle time par complexité	Du premier passage « en cours » au « terminé », par classe de poids (1–2, 3–5, 8–13)
Vitesse	Complexité livrée	La somme des poids terminés, divisée par les ETP de développement présents
Qualité	Taux de bugs	Les bugs liés à une story, rapportés à la complexité livrée ; en production et en préproduction
Qualité	Taux de reprise	La part des stories revenues de revue ou de QA vers « en cours »
Coût	Coût IA par point	La dépense en tokens de l'équipe, divisée par les points livrés
Expérience	Score d'expérience	Une enquête courte, de 1 à 5 : satisfaction, flow, charge cognitive, utilité de l'IA

Le taux de reprise est le signal le plus précoce d'une dette de qualité créée par l'IA : à surveiller dès le premier mois.

Les indicateurs de contexte, comme le run, le travail abandonné ou la ré-estimation, expliquent une variation sans constituer un gain : on les regarde pour comprendre, jamais pour afficher un progrès.

La courbe en J

Module 6 · slide « La courbe en J »

Pendant les premiers mois, le lead time ou la qualité peuvent se dégrader, le temps d'apprendre de nouvelles façons de travailler. Une baisse au deuxième mois n'est pas un échec ; une baisse qui dure au bout de plusieurs mois est un signal. On lit donc les objectifs sur les derniers sprints d'une période.

Le gain net

Module 6 · pour aller plus loin

Un gain se calcule net du coût IA : l'équipe livre plus de valeur qu'à sa référence, une fois déduit le coût IA supplémentaire, converti en valeur livrée. Seule la valeur compense un coût qui monte ; une économie de licences ne compte pas comme un gain.

Tenir l'outil de suivi propre

Module 6 · slide « Ce que l'équipe tient propre »

Sans ces champs, aucun des sept indicateurs ne se calcule, et la mesure de référence ne vaut rien.

- Le **poids final** de chaque story, pas seulement l'estimation de départ.
- Des **statuts tenus au fil de l'eau**, dès que le travail change d'état.
- Chaque **bug lié à la story** qui l'a introduit, avec l'environnement où on l'a trouvé (production, préproduction).
- Le **type** de chaque ticket : fonctionnalité, bug, reprise, dette.
- Le **coût IA attribué** à l'équipe et à la tâche : chaque appel passe par la passerelle avec son identité (module 5).
- L'**enquête d'expérience** remplie par toute l'équipe.

Le rythme de lecture

Module 6 · slide « Ce que l'équipe tient propre »

Quand	Quoi	Avec qui
Chaque semaine	La collecte des données	Automatique
Chaque mois	La calibration des gates ; les indicateurs relus	L'équipe
Chaque trimestre	L'enquête d'expérience ; la décision d'ouvrir ou non une vague	Le programme

> Mesurez l'équipe, jamais la personne.

Un indicateur qui sert à juger quelqu'un cesse d'être fiable.

Encore à caler

Module 6 · pendant le pilote

- **Les critères d'ouverture d'une vague** : quel écart, mesuré comment, décidé par qui.
- **Qui ratifie** qu'une étape avance d'un niveau de délégation.
- **La limite de relecture humaine** : jusqu'à quel volume un humain relit vraiment.
- **Une unité commune** de complexité entre les équipes.
- **Le coût par tâche**, pas encore attribué partout.

FICHE MODULE 7

Votre rôle de manager

Ce que fait un manager quand son équipe entre dans le nouveau cycle : les postures, les questions à poser, ce qu'on évite, trois risques à surveiller et la liste des 30 premiers jours.

Votre rôle de manager

Module 7 · slide « Votre rôle de manager »

Posture	Ce que ça veut dire	Pourquoi
Vous questionnez, l'équipe décide	Vous ne codez pas, n'activez pas un flag, ne signez pas à sa place	Sinon, la trace nomme quelqu'un qui n'a pas décidé, et l'équipe perd la main sur ses choix
Deux questions suffisent	« Qui l'a décidé ? Qui signe ? »	Elles révèlent ce qui manque : un responsable, un seuil, une source
Un NO-GO n'est pas une sanction	Il nomme un critère : l'équipe corrige, revérifie, redemande. Commentez aussi les GO : « vous passez parce que… »	Une gate qui renvoie avec un critère fait progresser ; une gate qui punit fait cacher
Quand ça coince, regardez le cadre	Le format, la gate, le contexte ; un profil qui manque se signale comme un écart à combler	On améliore le cadre ; on lit les résultats par équipe, jamais par personne
Pas de vitesse sans qualité	Un gain de vitesse ne compte que si la gate suivante passe au vert	Une vitesse payée en qualité n'est pas un gain

> L'agent propose, les humains décident.

Les questions à poser à l'équipe

Module 7 · slide « Votre rôle de manager »

- **Qui l'a décidé ? Qui signe ?**
- **Comment saurez-vous dans 30 jours que ça a marché ?** Et qui a fixé ce seuil ?
- **Ce test couvre quel critère d'acceptation ?**
- **Quelle version du contrat de l'autre équipe utilisez-vous ?**
- **Qui active la fonctionnalité, et comment revient-on en arrière ?**
- **Qu'est-ce qui est encore incertain ?** L'équipe écrit « à confirmer » ce qui n'est pas sûr, au lieu de le deviner.
- **Après un changement : la spec est-elle à jour avant le code ?**
- **Les critères de la gate étaient-ils clairs ?** Lequel l'était le moins ?

Quand l'équipe dit « ce n'est pas comme chez nous » : on le note, on ne le débat pas. Ces remarques font évoluer le cycle.

Ce qu'on évite

Module 7 · slide « Votre rôle de manager »

- Taper, activer un flag ou signer à la place de l'équipe.
- Répondre avant qu'on vous pose la question.
- Lever à la place de l'équipe une ambiguïté qu'elle doit faire remonter elle-même.
- Modifier le travail d'une autre équipe : on va la voir.
- Juger une personne sur un indicateur.
- Citer une vitesse sans la qualité qui va avec.
- Sauter le retour après une gate : il prend quelques secondes et améliore la gate suivante.

Trois risques à surveiller

Module 7 · slide « Trois risques à surveiller »

Risque	Le signal	Votre réflexe
La charge de relecture	Des relectures et des gates qui attendent	Découper plus petit ; protéger le créneau de relecture
L'approbation par réflexe	Une approbation proche de 100 %, des revues très courtes	Demander « qu'avez-vous refusé cette semaine ? » ; regarder le taux de reprise (module 6)
Les frictions de budget	Un budget de tokens dépassé, des sessions abandonnées	Lire le coût avec la qualité ; une conversation neuve avec l'agent après chaque gate (module 5)

- **La charge de relecture** : la relecture humaine tient jusqu'à un certain volume. Plus l'agent produit, plus elle devient le goulot.
- **L'approbation par réflexe** (le « tampon ») : quand presque tout est approuvé, on cesse de lire. Une étape AI-assisted devient alors AI-executed sans que personne l'ait décidé, et sans mesure.
- **Les frictions de budget** : on les attend au début, et on les lit au niveau de l'équipe, jamais par personne.

> La relecture tient jusqu'à un volume : au-delà, on ne lit plus.

Vos 30 premiers jours

Module 7 · slide « Vos 30 premiers jours »

Avant que les outils arrivent

Action	Module
Inscrire l'équipe entière, tous les rôles produit ensemble	4
Nommer les quatre casquettes, et connaître votre contact Evaluation Engineer	4
Vérifier que la mesure de référence est prise et l'outil de suivi propre	6
Vérifier les accès avant les formations	6

Pendant le premier mois

Action	Module
Garder un humain sur chaque tâche	1 et 2
Amorcer le Nexus par les Contracts	3
Annoncer que la mesure est collective, jamais individuelle	6
Prévoir, et dire à l'équipe, plus de relecture et des frictions de budget au début	5 et 7

> Protégez le temps de relecture et de décision.

Pour commencer : entourez une action à lancer dès lundi, et notez une question à rapporter au programme.

Encore à caler

Module 7 · pendant le pilote

- **La limite de la relecture humaine** : aucun critère ne dit encore jusqu'à quel volume un humain relit vraiment.
- **Détecter le tampon** : aucune méthode ne repère encore une relecture devenue automatique.
- **L'identité des agents en revue** : tant qu'un agent relit sous l'identité d'un humain, la trace nomme quelqu'un qui n'a pas décidé.

SUPPORT D'ACTIVITÉ

Les activités

Les activités de la formation, pour les refaire avec votre équipe : la consigne, le matériel et ce qu'on en retient.

Ranger dans le Nexus (4 min, optionnelle)

Module 3 · slide « Activité — Ranger dans le Nexus »

Objectif : placer six informations familières dans le Context Nexus, avec deux coordonnées chacune : un registre (Knowledge, Purpose, Contracts, Process) et un niveau (organisation, programme, équipe).

Déroulé

1. Par deux, recopiez la grille vide ci-dessous, ou notez simplement « n° → registre, niveau » (à distance, dans le chat : « 1 → Contracts, organisation »).
2. Placez les six cartes en 2 minutes. Une carte peut aller dans plusieurs cases, ou dans aucune.
3. Correction carte par carte, puis débrief.

Les cartes (chez Vélora, groupe hôtelier fictif)

1. « Toutes nos interfaces respectent le niveau WCAG AA. »
2. Le glossaire partagé par les équipes du programme.
3. « Rappel de départ : –x % de départs non annoncés à la réception. »
4. Le runbook « VelNotif ne répond plus ».
5. L'ADR « on passe par VelNotif, pas par un prestataire d'envoi externe ».
6. « Sur un écran de suivi, l'équipe accepte 400 ms de temps de réponse au lieu des 200 ms fixés par le groupe. »

La grille

	Knowledge	Purpose	Contracts	Process
Organisation				
Programme				
Équipe				

Pour vous aider : le handout « Le Context Nexus », sections « Les quatre registres », « Découper une ADR », « La grille registres × niveaux » et « Étendre, déroger, contredire ».

À retenir

- Un registre dit ce qu'est une information ; un niveau dit jusqu'où elle s'applique.
- Une ADR ne se range pas d'un bloc.
- Une équipe ne contredit pas une règle du groupe : elle déroge en le déclarant, avec l'approbation du niveau au-dessus.

Avec votre équipe : lesquelles de ces six informations existent déjà par écrit chez vous, à quel niveau, et où ?

Distribuer les casquettes (8 min)

Module 4 · slide « Activité — Distribuer les casquettes »

Objectif : nommer un porteur pour chaque casquette de votre équipe, voir qui porte trop, et choisir un levier.

Déroulé

1. Notez les membres de votre équipe, avec leurs initiales seulement : on regarde l'équipe, on ne juge personne.
2. Placez un porteur sur chaque casquette (4 minutes, seul).
   - Une personne peut porter plusieurs casquettes.
   - L'Orchestrator peut tourner entre 2 ou 3 personnes : partagez sa charge.
   - Le Decision Guarantor est une seule personne, nommée.
3. Additionnez la charge indicative par personne.
4. Entourez la personne la plus chargée et choisissez un levier : répartir, faire tourner, ou signaler un écart (un profil qui manque).
5. Comparez à deux (2 minutes), puis débrief.

La grille

Casquette	Qui ?	Charge indicative
Context Engineer		~0,25 ETP
Specification Engineer		~0,35 ETP
Agentic Workflow Orchestrator		~0,3 ETP
Decision Guarantor		~0,15 ETP
Notre contact Evaluation Engineer (pool)		—

Pour vous aider : le handout « Équipe et nouveaux rôles », sections « Les quatre casquettes de l'équipe » et « Le premier goulot à repérer ».

À retenir

- Nommer les casquettes avant que l'outillage n'arrive : sinon, ce sont les outils qui décident qui fait quoi.
- La première surcharge est souvent la personne la plus sollicitée aujourd'hui.
- Une casquette que personne ne peut porter est un écart à remonter, pas une faute.

Avec votre équipe : refaites la grille ensemble, et demandez à chaque personne si sa charge lui paraît réaliste.

Gain ou pas ? (3 min)

Module 6 · slide « Exercice éclair — Gain ou pas ? »

Objectif : lire des indicateurs comme un manager, la vitesse avec la qualité, le coût et l'expérience, sans conclure trop vite.

Déroulé

1. Lisez les trois équipes, comparées à leur mesure de référence.
2. Pour chacune, tranchez : « oui », « pas encore » ou « attendre » (à distance, dans le chat : « A … , B … , C … »).
3. Correction, puis débrief.

Les trois équipes

Équipe	Ce qu'on observe, par rapport à la référence
A	Lead time −25 % ; taux de reprise +40 %
B	Lead time +10 % au deuxième mois ; bugs stables ; enquête d'expérience en hausse
C	Complexité livrée +15 % ; bugs stables ; coût IA par point en baisse

Pour vous aider : le handout « Déploiement et mesure », sections « Mesurer la valeur, pas l'adoption », « Les sept indicateurs » et « La courbe en J ».

À retenir

- Une vitesse ne se lit jamais seule : la qualité et le coût vont avec.
- Une baisse au début est attendue ; une baisse qui dure est un signal.
- Sans mesure de référence prise avant l'outillage, aucun gain ne se démontre.

Avec votre équipe : quelle donnée vous manquerait aujourd'hui pour trancher ?

DÉMOS À REJOUER

Les démos

Les démos jouées par le formateur pendant la formation, pour les rejouer en solo ou avec votre équipe, que vous ayez assisté à la session ou non. Elles ne demandent qu'un assistant de chat.

Avec ou sans contexte

Module 3 · démo « Avec ou sans contexte » · 3 min

Ce que ça montre : la même demande, faite à la même IA, sans puis avec le contexte de l'équipe. Sans contexte, l'IA produit du code propre et plausible, mais elle invente tout ce qui relève de nos décisions. Avec une fiche d'une quinzaine de lignes, elle les reprend. L'IA est bonne en général ; elle ne connaît pas nos décisions.

Le cas : chez Vélora, groupe hôtelier fictif, l'équipe qui gère les messages aux clients veut envoyer un rappel la veille du départ.

À rejouer

1. Ouvrez une conversation vide et collez la demande seule :

```text
Écris la fonction qui envoie au client un rappel la veille de son départ.
En 25 lignes au plus.
```

2. Ouvrez une deuxième conversation vide. Collez d'abord la fiche de contexte, puis la même demande :

```text
Contexte de l'équipe Notifications, groupe hôtelier Vélora.

Ce qu'on sait
- Les messages clients passent par le service interne VelNotif : notification push dans l'app
- L'heure qui fait foi est l'heure locale de l'hôtel.
- La langue du message est celle de la réservation.

Ce qu'on veut
- Réduire les départs non annoncés à la réception ; cible fixée par le produit.

Ce qu'on exige
- API VelNotif v2 : POST /messages avec reservationId, templateId, locale.
- Jamais d'envoi entre 21h et 8h, heure locale de l'hôtel.
- Un seul rappel par réservation : clé d'idempotence = reservationId + "rappel-depart".

Ce qu'on fait
- En cas d'échec de VelNotif : deux nouvelles tentatives au plus, puis on journalise.
- Le texte du message est validé par le produit avant mise en ligne ; le code utilise son tem
- Un interrupteur (feature flag) coupe l'envoi.
```

3. Comparez les deux réponses sur ces six points :

Point	Sans contexte	Avec contexte
Canal et service d'envoi		

Point	Sans contexte	Avec contexte
Heure d’envoi		
Langue du message		
Plage « ne pas déranger »		
Doublons (deux rappels pour la même réservation)		
Qui valide le texte du message		

Ce qu’on observe : sans contexte, l’IA choisit un prestataire d’envoi connu, l’heure du serveur, une langue au hasard, écrit le texte du message elle-même et ne protège pas contre les doublons. Avec la fiche, elle appelle VelNotif, calcule l’heure locale de l’hôtel, respecte la nuit, utilise la clé d’idempotence et le template validé. Un modèle récent écrit du bon code générique dans les deux cas : tout l’écart vient de nos décisions.

Pour aller plus loin : la fiche est déjà rangée en quatre blocs, les quatre registres du Context Nexus (handout « Le Context Nexus »). Retirez un bloc et relancez : ce qui disparaît de la réponse, c’est ce que l’agent ne pouvait pas deviner.

GLOSSAIRE

Glossaire

Une phrase par terme, dans l'ordre où la formation les introduit. Les termes anglais sont gardés : ce sont ceux que vos équipes liront dans les outils et les documents du programme.

Le cycle et la délégation

Modules 0 et 1

Terme	En une phrase
PDLC (Product Development Life Cycle)	Le cycle de vie d'un produit, de l'idée à son retrait, en six phases : Discover, Design, Develop, Deploy, Deepen, Drop.
Phase	Une des six étapes de la roue ; chacune remet une spec à la suivante.
Spec	Le document qui décrit l'intention, les critères d'acceptation et les contrats : l'unité de travail du nouveau cycle.
Spec-Driven Development (SDD)	Travailler à partir de specs relues et signées : l'agent produit le code à partir de la spec, pas d'une demande orale.
Critère d'acceptation	Une condition vérifiable qu'une livraison doit remplir ; chacun a son test.
Agent	Un outil d'IA qui enchaîne lui-même des actions (lire, écrire, lancer des tests) pour atteindre un objectif.
Human	Niveau de délégation : une personne fait le travail et décide.
AI-assisted	Niveau de délégation : l'IA produit, une personne voit chaque sortie et décide. Le niveau de tout le pilote.
AI-executed	Niveau de délégation : l'IA produit, un contrôle automatique décide ; un humain voit un échantillon et chaque échec.
AI-autonomous	Niveau de délégation : l'IA agit, un contrôle décide, l'humain ne voit que les exceptions ; encore en débat.
Rampe de délégation	La règle qui fait gagner un niveau à une activité sur résultats mesurés, et la fait redescendre si la qualité baisse.
Périmètre sensible	Le code et les changements à risque (paiement, données réglementées) : ils restent en AI-assisted, approuvés par un humain.
Pilote	Les premiers mois du programme, avec quelques équipes entières, avant le passage à l'échelle.

Les quality gates

Module 2

Terme	En une phrase
Quality gate	Un point de contrôle où l'on vérifie des preuves avant de passer à la suite ; verdict GO ou NO-GO.
Gate de phase	Une gate au passage d'une phase à l'autre (G1 à G6), tenue par un humain nommé.
Gate de tâche	Une gate entre deux étapes d'une tâche confiée à un agent, plusieurs fois par jour, surtout automatique.
Gate-check	Le contrôle automatique d'une gate : des critères chiffrés, que l'équipe peut lancer elle-même avant son passage.
Gate keeper	La personne nommée qui décide GO ou NO-GO sur ce que la machine ne voit pas.
DoR (Definition of Ready)	Ce qui doit être prêt avant de commencer à construire ; la gate G2 la vérifie.
DoD (Definition of Done)	Ce qui doit être vrai pour qu'un travail soit fini ; la gate G3 la vérifie.
NFR (exigences non fonctionnelles)	Les exigences de performance, de disponibilité, d'accessibilité ou de rétention, toujours chiffrées.
Feature flag	Un interrupteur qui active ou coupe une fonctionnalité déjà en production.
Release ≠ Activation	Mettre en production et activer sont deux décisions humaines distinctes, tracées et réversibles.
Runbook	La procédure écrite pour exploiter un service et réagir à un incident.
Calibration	La revue mensuelle des gates : taux de passage, taux d'échappement, seuils à ajuster.
Taux d'échappement	La part de ce qui a passé une gate et a cassé plus tard.

Le Context Nexus

Module 3

Terme	En une phrase
Context Nexus	Le contexte partagé de l'équipe, lisible par les humains et par les agents : ce qu'elle sait, veut, exige et fait.
Registre	Une des quatre familles du Nexus : Knowledge, Purpose, Contracts, Process.
Knowledge	Ce qu'on sait : glossaire, architecture, conventions, Decision Contexts.

Terme	En une phrase
Purpose	Ce qu'on veut : specs, critères d'acceptation, Decision Directives, objectifs ; écrit par des humains.
Contracts	Ce qu'on exige : des règles vérifiables avec un seuil (DoD, sécurité, contrats d'API) ; écrites par des humains.
Process	Ce qu'on fait : runbooks, déploiement, retour arrière.
Niveau	La portée d'une information : organisation, programme ou équipe ; une règle du groupe vaut pour toutes les équipes.
Exception	Une dérogation d'équipe à une règle du niveau au-dessus, déclarée et approuvée ; sinon l'agent ne la voit pas.
ADR (Architecture Decision Record)	La trace écrite d'une décision d'architecture ; dans le Nexus, elle se découpe en deux artefacts.
Decision Context	La partie « pourquoi » d'une décision : situation, options, raisons ; rangée dans Knowledge.
Decision Directive	La partie « règle » d'une décision, courte et lue par l'agent à chaque tâche ; rangée dans Purpose.
Contrat d'API	La description publiée de ce qu'un service offre aux autres équipes, versionnée.
Version fixée (épinglée)	La version d'un contrat qu'une équipe déclare utiliser.
Stale	Le statut d'une spec ou d'un contexte qui dépend d'une version dépassée : à revoir avant de coder.
Task brief	Le contexte assemblé pour une tâche précise : ce que l'agent reçoit avant de travailler.

L'équipe et les rôles

Module 4

Terme	En une phrase
Casquette	Un rôle porté par une personne déjà dans l'équipe, en plus ou à la place d'une partie de son travail.
Context Engineer	Tient le contexte que lit l'agent : sourcé, daté, réutilisable.
Specification Engineer	Transforme l'intention produit en spec vérifiable.
Agentic Workflow Orchestrator	Découpe le travail, lance les agents, place les relectures, suit le budget de tokens ; tourne entre 2 ou 3 personnes.

Terme	En une phrase
Decision Guarantor	Signe, en son nom, ce qui part en production ; une personne nommée et stable.
Evaluation Engineer	Vérifie dans la durée la qualité de ce que produisent les agents ; partagé entre plusieurs équipes.
AI Champion	Un membre d'une équipe qui diffuse les pratiques et fait remonter les retours.
ETP (équivalent temps plein)	L'unité des charges indicatives : 0,25 ETP, c'est un quart du temps d'une personne.

La toolkit

Module 5

Terme	En une phrase
Claude Code	L'agent de code de référence du programme.
Spec Kit	L'outil de Spec-Driven Development : de la constitution aux tâches, puis au code.
LiteLLM	La passerelle unique vers les modèles : quotas, identités, traces et coûts.
Passerelle (gateway)	Le point d'entrée par lequel passe chaque appel aux modèles.
Classe de données	La sensibilité des données d'un appel (publiques, internes, confidentielles, réglementées) ; elle fixe les modèles permis.
Tokens	L'unité de compte des modèles : ce qu'on envoie et reçoit, et ce qu'on paie.
Budget de tokens	Le plafond de consommation d'une exécution ; le dépasser est un NO-GO.
Hook	Un contrôle automatique exécuté avant une action de l'agent, qui bloque et trace ce qu'un prompt ne garantit pas.
Permission	La liste versionnée de ce que l'agent peut lancer, et de ce qui lui est interdit.
CI (intégration continue)	Les contrôles automatiques lancés à chaque changement de code : lint, tests, scans.

Le déploiement et la mesure

Modules 6 et 7

Terme	En une phrase
Vague	Un groupe d'équipes qui entre dans le nouveau cycle en même temps, après le pilote.

Terme	En une phrase
Mesure de référence (baseline)	Le point de départ de chaque équipe, mesuré avant l'outillage avec les mêmes indicateurs qu'après.
Lead time	Le temps entre la création d'un besoin et sa livraison.
Cycle time	Le temps entre le premier passage « en cours » d'une story et son « terminé », comparé à complexité égale.
Complexité livrée	La somme des poids des stories terminées, rapportée à la taille de l'équipe.
Taux de reprise	La part des stories revenues « en cours » après une revue ou la QA.
Coût IA par point	La dépense en tokens de l'équipe, divisée par les points de complexité livrés.
Courbe en J	La baisse attendue des premiers mois, avant que l'équipe dépasse sa référence.
Indicateur de contexte	Run, travail abandonné, ré-estimation : ils expliquent une variation sans compter comme un gain.
Gain net	La valeur livrée en plus de la référence, une fois déduit le coût IA supplémentaire.
Approbation par réflexe (le « tampon »)	Une relecture devenue automatique : on approuve tout, on ne lit plus rien.